

WESAD - Tutto quello che resta da fare

Roadmap completa per finire il progetto: da S2+BVP funzionante fino a tutti i soggetti, modelli, metriche, confronto con il paper, grafici, slide e consegna finale.

Obiettivo del documento

Questo documento parte dallo stato attuale del progetto: avete già letto S2.pkl, estratto il BVP dal polso, creato finestre da 60 secondi, calcolato feature semplici e generato `bvp_features_s2.csv` con una soglia di purezza della label al 70%. Da qui in avanti l'obiettivo è trasformare questa prima prova in un progetto completo, ripetibile e confrontabile con il benchmark WESAD.

La regola di lavoro deve restare incrementale: un segnale, un modello, una pipeline funzionante. Solo dopo si passa al segnale successivo.

Indice operativo

1. Stato attuale e risultato già ottenuto
2. Obiettivo finale del progetto
3. Folder structure definitiva
4. Pulizia e stabilizzazione del codice
5. Estendere la feature extraction da S2 a tutti i soggetti
6. Validare il CSV finale prima del modello
7. Primo modello: BVP wrist + kNN
8. Valutazione corretta: Leave-One-Subject-Out
9. Metriche da produrre e come leggerle
10. Debug se i risultati non si avvicinano alla tabella del paper
11. Secondo esperimento: EDA wrist + kNN
12. Esperimento opzionale: BVP + EDA
13. Parte chest: RESP, ECG, EMG con Random Forest
14. Decision Tree opzionale e confronto tra modelli
15. Tabelle finali, grafici e interpretazione
16. Confronto con la letteratura WESAD
17. Limiti metodologici da dichiarare
18. Struttura della presentazione finale
19. Organizzazione della consegna
20. Checklist finale

1. Stato attuale e risultato già ottenuto

Ad oggi il progetto non è più solo teorico: avete una prima pipeline Python che funziona su S2 e sul segnale BVP da polso. Questo è il punto di partenza da non rompere.

Elemento	Stato attuale	Perché è importante
Ambiente	Python installato, .venv attivo, librerie installate	Permette di lavorare in modo isolato e riproducibile
Lettura dati	S2.pkl letto con pickle.load	Conferma che sappiamo aprire correttamente i file WESAD
Segnale	BVP wrist estratto	Primo segnale scelto per BVP + kNN
Window	60s con step 30s	Coerente con la logica del paper e gestibile
Label	1 e 3 trasformate in non-stress, 2 in stress	Task binario stress vs non-stress
Purezza	Finestra tenuta solo se label prevalente >= 70%	Riduce finestre miste e label rumorose
Output	bvp_features_s2.csv	Prima tabella di feature pronta per ML

Da ricordare

Il modello non riceverà la label come input. Riceverà solo le feature del segnale: bvp_mean, bvp_std, bvp_min, bvp_max, bvp_range. La label serve solo come y, cioè risposta corretta per addestramento e valutazione.

2. Obiettivo finale del progetto

L'obiettivo finale non è creare un sistema industriale real-time. L'obiettivo corretto è costruire una pipeline di Machine Learning supervisionato per classificare stress vs non-stress su WESAD, confrontando i risultati con il benchmark del paper.

Output finale	Descrizione
CSV di feature	File tabellari per ogni segnale o combinazione di segnali, costruiti da tutti i soggetti
Modelli addestrati	kNN per wrist BVP/EDA; Random Forest per chest RESP/ECG/EMG
Metriche	Accuracy, F1-score, precision, recall, confusion matrix, risultati per soggetto
Confronto	Tabella vostra vs Tabella 4 del paper WESAD sul task binario
Presentazione	Massimo 15 slide, sintetica ma completa
Codice	Cartella ordinata con script e README

Task principale

Task: stress vs non-stress

Label WESAD originali:

```
1 = baseline    -> non-stress -> 0
2 = stress      -> stress    -> 1
3 = amusement   -> non-stress -> 0
0, 4, 5, 6, 7   -> scartate
```

3. Folder structure definitiva

Prima di espandere il progetto, conviene organizzare bene la cartella. Questo evita confusione quando inizierete ad avere più script, più CSV e più risultati.

```
WESAD_Project/
|-- .venv/
|-- data_raw/
|   |-- S2/
|   |-- S3/
|   |-- S4/
|   `-- ...
|-- data_features/
```

```
| |-- bvp_features_all_60s.csv
| |-- eda_features_all_60s.csv
| |-- resp_features_all_60s.csv
| |-- ecg_features_all_60s.csv
| |-- emg_features_all_60s.csv
|-- results/
| |-- bvp_knn_results.csv
| |-- eda_knn_results.csv
| |-- resp_rf_results.csv
| |-- summary_results.csv
|-- figures/
| |-- confusion_matrix_bvp_knn.png
| |-- bar_accuracy_f1.png
| |-- per_subject_scores.png
|-- src/
| |-- read_subject.py
| |-- extract_features.py
| |-- train_knn_loso.py
| |-- train_rf_loso.py
| |-- utils.py
|-- notebooks/          # opzionale
|-- README.md
|-- requirements.txt
```

Se non volete spostare subito tutto, almeno create `data_features`, `results` e `figures`. I file raw possono restare nelle cartelle S2, S3, ecc., ma e' meglio avere una struttura pulita prima della consegna.

4. Pulizia e stabilizzazione del codice

Prima di passare a tutti i soggetti, bisogna trasformare lo script attuale da 'codice che funziona su S2' a 'funzione riutilizzabile'.

Da fare	Dettaglio operativo	Controllo
Togliere hardcoding	Non scrivere solo <code>Path('S2') / 'S2.pkl'</code> . Passare <code>subject_folder</code> come parametro	Lo script deve funzionare anche con S3, S4, ecc.
Creare funzione	<code>create_bvp_features_for_subject(subject_folder)</code>	Ritorna un DataFrame
Tenere parametri in alto	<code>window_sec=60, step_sec=30, purity_threshold=0.70</code>	Facile cambiare configurazione
Rimuovere feature non scelte	Per ora usare mean, std, min, max, range	Pipeline piu' semplice e difendibile
Salvare CSV	Scrivere i CSV in <code>data_features</code>	Non sporcare la root del progetto
Gestire warning	Il <code>VisibleDeprecationWarning</code> non blocca il progetto	Ignorabile finche' i dati vengono letti

Feature BVP consigliate per la baseline

```
bvp_mean
bvp_std
bvp_min
bvp_max
bvp_range = bvp_max - bvp_min
```

Median ed energy possono essere aggiunte dopo, ma per il primo confronto con kNN e' meglio partire con poche feature semplici, per ridurre ridondanza e rischio di overfitting.

5. Estendere la feature extraction da S2 a tutti i soggetti

Questo e' il prossimo vero passo del progetto. S2 serve solo per debug. Per addestrare un modello sensato bisogna creare il CSV BVP su tutti i soggetti disponibili.

Regola fondamentale: caricare un soggetto alla volta, estrarre feature, aggiungere al DataFrame finale, poi liberare memoria. Non caricare tutti i .pkl insieme.

Soggetti da usare

WESAD ha 15 soggetti validi. S1 e S12 sono esclusi/mancanti per problemi ai sensori. Quindi normalmente si usano le cartelle disponibili: S2, S3, S4, S5, S6, S7, S8, S9, S10, S11, S13, S14, S15, S16, S17.

Pseudo-codice

```
all_rows = []

for subject_folder in subject_folders:
    data = load_pickle(subject_folder / f"{subject}.pkl")
    df_subject = create_bvp_features(data)
    all_rows.append(df_subject)
    del data

final_df = pd.concat(all_rows, ignore_index=True)
final_df.to_csv("data_features/bvp_features_all_60s.csv",
               index=False)
```

Cosa deve contenere il CSV finale BVP

Colonna	Tipo	Uso
subject	string	Serve per Leave-One-Subject-Out
start_sec, end_sec	int	Tracciabilita' temporale della finestra
window_sec, step_sec	int	Documentano la configurazione usata
bvp_mean, bvp_std, bvp_min, bvp_max, bvp_range	float	Feature usate come input X
original_label	int	Label WESAD originale: 1, 2, 3
label	int	Target binario: 0 non-stress, 1 stress
label_purity	float	Quota della label prevalente nella finestra

6. Validare il CSV finale prima del modello

Prima di addestrare kNN, bisogna controllare il CSV. Questa fase evita errori difficili da scoprire dopo.

Controllo	Comando/criterio	Cosa deve succedere
Shape	df.shape	Deve avere righe da tutti i soggetti, non solo S2
Soggetti	df['subject'].nunique()	Deve essere circa 15
Label	df['label'].value_counts()	Devono esserci sia 0 sia 1
Original label	df['original_label'].value_counts()	Devono esserci 1, 2, 3
Purezza	df['label_purity'].min()	Deve essere >= 0.70
NaN	df.isna().sum()	Non devono esserci NaN nelle feature
Distribuzione per soggetto	groupby subject e label	Ogni soggetto dovrebbe avere righe utili

Codice rapido di controllo

```
import pandas as pd

df = pd.read_csv("data_features/bvp_features_all_60s.csv")

print(df.shape)
print(df["subject"].nunique())
print(df["label"].value_counts())
print(df["original_label"].value_counts())
print(df["label_purity"].min())
print(df.isna().sum())
print(df.groupby(["subject", "label"]).size())
```

7. Primo modello: BVP wrist + kNN

Quando il CSV BVP su tutti i soggetti e' pronto, si passa al primo modello vero. Il primo esperimento deve restare semplice: BVP wrist + kNN, task binario stress vs non-stress.

Parametro	Scelta consigliata	Motivo
Segnale	BVP wrist	Primo segnale scelto e gia' testato su S2
Feature	mean, std, min, max, range	Baseline semplice
Modello	kNN	Coerente con la vostra scelta e con la tabella del paper
k	9	Nel paper kNN e' usato con k=9
Scaler	StandardScaler	kNN usa distanze, quindi serve normalizzare
Validazione	Leave-One-Subject-Out	Evita leakage tra finestre dello stesso soggetto

Colonne da usare come X e y

```
feature_cols = [
    "bvp_mean",
    "bvp_std",
    "bvp_min",
    "bvp_max",
    "bvp_range",
]

X = df[feature_cols]
y = df["label"]
groups = df["subject"]
```

Non inserire mai `original_label`, `label`, `label_purity`, `start_sec` o `end_sec` dentro X. Sono metadati o target, non feature fisiologiche.

8. Valutazione corretta: Leave-One-Subject-Out

La valutazione corretta per WESAD non e' `train_test_split` casuale sulle righe. Le righe dello stesso soggetto sono simili tra loro; uno split casuale farebbe finire finestre dello stesso soggetto sia nel training sia nel test, dando risultati troppo ottimistici.

Leave-One-Subject-Out significa: a ogni giro si lascia fuori un soggetto intero come test set, e si allena sugli altri soggetti. Alla fine si fa la media delle metriche.

Struttura del training LOSO

```
from sklearn.model_selection import LeaveOneGroupOut
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, f1_score

logo = LeaveOneGroupOut()

for train_idx, test_idx in logo.split(X, y, groups):
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    model = KNeighborsClassifier(n_neighbors=9)
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)

    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
```

Errore da evitare

```
# SBAGLIATO: normalizza tutto prima della validazione
X_scaled = scaler.fit_transform(X)

# GIUSTO: fit dello scaler solo sul training fold
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

9. Metriche da produrre e come leggerle

L'accuracy da sola non basta. La consegna del progetto chiede una valutazione non limitata all'accuratezza; inoltre WESAD ha classi non perfettamente bilanciate. Per questo bisogna produrre almeno accuracy, precision, recall, F1-score e confusion matrix.

Metrica	Cosa misura	Perche' serve
Accuracy	Quota totale di predizioni corrette	Facile da confrontare con la tabella del paper
Precision	Quando il modello dice stress, quante volte ha ragione	Riduce falsi allarmi
Recall	Quanto stress reale viene trovato	Importante se non vogliamo perdere casi di stress
F1-score	Media armonica tra precision e recall	Buona metrica sintetica per classi sbilanciate
Confusion matrix	Mostra veri/falsi positivi e negativi	Fa capire dove sbaglia il modello
Per-subject score	Metriche per ciascun soggetto lasciato fuori	Mostra se alcuni soggetti sono difficili

Output da salvare

```
results/bvp_knn_loso_per_subject.csv
results/bvp_knn_summary.txt
figures/confusion_matrix_bvp_knn.png
figures/per_subject_bvp_knn.png
```

10. Debug se i risultati non si avvicinano alla tabella del paper

Il target del paper per BVP + kNN nel task binario e' circa accuracy 82.06% e F1-score 78.94%. Non e' obbligatorio ottenere identico, perche' la nostra feature extraction e' semplificata. Pero' se i risultati sono molto bassi, bisogna fare debug metodico.

Problema	Possibile causa	Cosa controllare
Accuracy molto bassa	Label sbagliate	Verificare mapping: 1/3 -> 0, 2 -> 1
F1 molto bassa	Il modello predice quasi sempre non-stress	Controllare class imbalance e confusion matrix
Risultati troppo alti	Leakage	Controllare di non avere subject nel training e test insieme
Errore su scaling	Scaler applicato male	Fit solo sul train fold
Poche righe	Soggetti non caricati o troppe finestre scartate	Controllare subject count e label_purity
Valori NaN	Feature calcolate su finestra vuota o dati sporchi	df.isna().sum()
Differenza dal paper	Feature troppo semplici	Spiegare che non si usano HR/HRV avanzate

Cosa NON fare per inseguire il numero

- Non fare train_test_split casuale per ottenere accuracy piu' alta.
- Non mettere label, original_label, label_purity o subject nelle feature X.
- Non cambiare mille parametri insieme: se qualcosa cambia, deve cambiare una cosa alla volta.
- Non dichiarare che il modello e' real-time se la segmentazione usa le label vere del dataset.

11. Secondo esperimento: EDA wrist + kNN

Quando BVP + kNN funziona su tutti i soggetti, si passa al secondo segnale da polso: EDA wrist + kNN. La pipeline deve restare quasi identica: stesso task, stessa validazione, stesso modello, stessa logica di windowing.

Elemento	BVP	EDA
Device	Wrist	Wrist
Sampling rate	64 Hz	4 Hz
Window	60s	60s
Step	30s	30s
Modello	kNN k=9	kNN k=9
Feature base	mean, std, min, max, range	mean, std, min, max, range, slope opzionale

Per EDA il numero di valori per finestra e' molto piu' basso

```
EDA wrist = 4 Hz
Window = 60s
Valori per finestra = 4 * 60 = 240 valori
```

L'EDA e' piu' lenta del BVP, ma fisiologicamente molto utile per lo stress perche' misura la conduttanza cutanea/sudorazione, legata all'attivazione del sistema nervoso simpatico.

Feature EDA consigliate

Feature	Formula/idea	Obbligatoria?
eda_mean	Media della finestra	Si
eda_std	Deviazione standard	Si
eda_min	Minimo	Si
eda_max	Massimo	Si
eda_range	max - min	Si
eda_slope	pendenza approssimata tra inizio e fine	Opzionale ma utile

Target indicativo dal paper

Per EDA wrist + kNN nel task binario, la tabella del paper riporta circa accuracy 73.13% e F1-score 68.30%. Usatelo come riferimento, non come numero obbligatorio.

12. Esperimento opzionale: BVP + EDA

Dopo BVP singolo ed EDA singolo, una prova molto sensata e' combinare i due segnali wrist. Questo permette di ragionare sulla sensor fusion: cuore/PPG piu' sudorazione/EDA.

La combinazione va fatta per finestra temporale, non per indice raw. Per ogni finestra 60s si calcolano feature BVP e feature EDA usando gli stessi start_sec e end_sec.

Feature finali BVP + EDA	Origine
bvp_mean, bvp_std, bvp_min, bvp_max, bvp_range	BVP wrist
eda_mean, eda_std, eda_min, eda_max, eda_range, eda_slope	EDA wrist
label, subject, label_purity	Metadati/target

Perche' ha senso

- BVP cattura risposte cardiovascolari.
- EDA cattura attivazione/sudorazione.
- I due segnali possono contenere informazione complementare.
- Confrontando BVP, EDA e BVP+EDA si capisce se la fusione migliora davvero.

13. Parte chest: RESP, ECG, EMG con Random Forest

Dopo aver completato la parte wrist, si passa ai segnali chest. Qui il modello scelto e' Random Forest, piu' adatto e robusto rispetto a un singolo Decision Tree.

Esperimento	Segnale	Modello	Target indicativo paper
C1	RESP chest	Random Forest	Accuracy circa 86.63%, F1 circa 84.33%
C2	ECG chest	Random Forest	Accuracy circa 82.78%, F1 circa 79.64%
C3	EMG chest	Random Forest	Accuracy circa 63.44%, F1 circa 49.42%
C4 opzionale	ECG + EMG + RESP	Random Forest	Da confrontare con segnali singoli

Frequenza e finestre

I segnali chest nel .pkl hanno la stessa frequenza delle label: 700 Hz. Quindi per una finestra da 60 secondi avete 42.000 valori per segnale. Questo e' normale e non va dato direttamente al modello: si estraggono feature riassuntive.

```
Chest signal = 700 Hz
Window = 60s
Valori per finestra = 700 * 60 = 42000
```

Feature semplici per i segnali chest

Segnale	Feature baseline	Nota interpretativa
RESP	mean, std, min, max, range	Respirazione: probabile segnale forte, ma attenzione al parlato durante TSST
ECG	mean, std, min, max, range	Versione semplice; HR/HRV sarebbero piu' vicine al paper ma piu' complesse
EMG	mean, std, min, max, range	Attivita' muscolare; potrebbe essere piu' rumoroso e meno performante

Random Forest consigliata

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    min_samples_split=20,
    criterion="entropy",
    random_state=42,
    n_jobs=-1
)
```

Per Random Forest la normalizzazione non e' necessaria come per kNN, ma la validazione per soggetto resta obbligatoria.

14. Decision Tree opzionale e confronto tra modelli

Decision Tree puo' essere usato come baseline interpretabile, ma non lo sceglierei come modello principale. Random Forest e' piu' stabile perche' combina molti alberi.

Modello	Pro	Contro	Uso consigliato
Decision Tree	Semplice da spiegare e visualizzare	Instabile, tende a overfittare	Baseline opzionale
Random Forest	Robusta, di solito performa meglio	Meno immediata da interpretare	Modello principale chest
kNN	Semplice, coerente con parte wrist	Sensibile a scaling e feature inutili	Modello principale wrist

Regola di confronto

Confrontate i segnali dentro lo stesso blocco e con lo stesso modello. Per esempio: BVP vs EDA con kNN; RESP vs ECG vs EMG con Random Forest. Se confrontate BVP+kNN con RESP+RF, dichiarate che cambia sia il segnale sia il classificatore.

15. Tabelle finali, grafici e interpretazione

Alla fine il progetto deve produrre risultati leggibili, non solo output nel terminale. Servono tabelle e grafici chiari.

Tabella principale consigliata

Area	Segnale	Modello	Accuracy	F1-score	Precision	Recall
Wrist	BVP	kNN	...	...	...	...
Wrist	EDA	kNN	...	...	...	...
Wrist	BVP + EDA	kNN	...	...	...	...
Chest	RESP	RF	...	...	...	...
Chest	ECG	RF	...	...	...	...
Chest	EMG	RF	...	...	...	...
Chest	ECG + EMG + RESP	RF	...	...	...	...

Grafici minimi

Grafico	Cosa mostra	Perche' serve
Bar chart Accuracy/F1	Performance per esperimento	Confronto immediato
Confusion matrix	Errori stress/non-stress	Capire falsi positivi e falsi negativi
Per-subject score	Accuracy/F1 per soggetto lasciato fuori	Mostra variabilita' individuale
Label distribution	Numero righe per classe	Controllo dataset

Interpretazione attesa

- BVP dovrebbe performare meglio di EDA con kNN, in linea con il paper.
- RESP dovrebbe essere uno dei segnali chest piu' forti con Random Forest.
- EMG potrebbe essere piu' debole e rumoroso.
- Le combinazioni possono migliorare, ma non sempre: se aggiungono rumore, possono peggiorare.

16. Confronto con la letteratura WESAD

Nel progetto dovete sempre distinguere tra replica del paper e vostra pipeline semplificata. Il paper usa feature piu' ricche, in particolare per ECG/BVP usa HR e HRV; per EDA considera componenti e picchi; per RESP considera feature specifiche della respirazione. Voi potete partire da feature semplici, ma dovete dichiararlo.

Tabella di confronto da inserire

Esperimento	Paper - Accuracy	Paper - F1	Vostro - Accuracy	Vostro - F1	Commento
BVP + kNN	82.06	78.94	...	...	Differenza dovuta a feature semplici/windowing
EDA wrist + kNN	73.13	68.30	...	...	Confronto dentro wrist
RESP + RF	86.63	84.33	...	...	Segnale chest forte
ECG + RF	82.78	79.64	...	...	Feature HR/HRV mancanti possono pesare

Esperimento	Paper - Accuracy	Paper - F1	Vostro - Accuracy	Vostro - F1	Commento
EMG + RF	63.44	49.42	...	...	Segnale piu' debole

Frase corretta da usare

L'obiettivo e' replicare in modo semplificato il benchmark WESAD sul task binario stress vs non-stress. Eventuali differenze rispetto alla Tabella 4 sono discusse considerando feature extraction, finestra temporale, soglia di purezza, normalizzazione e validazione.

17. Limiti metodologici da dichiarare

I limiti non indeboliscono il progetto; lo rendono piu' credibile. Dovete mostrarli nella presentazione finale.

Limite	Spiegazione	Come scriverlo bene
Dataset piccolo	15 soggetti validi	I risultati indicano fattibilita', non generalizzazione universale
Laboratorio	Stress indotto con TSST	Non equivale a stress spontaneo nel mondo reale
Poche feature	Baseline semplice	Scelta coerente con il livello del corso
Finestre sovrapposte	Righe non indipendenti	Per questo si usa Leave-One-Subject-Out
Segmentazione label-aware	La purezza usa label vere	Valida offline, non direttamente real-time
RESP e parlato	Durante TSST i soggetti parlano	RESP potrebbe catturare parlato oltre allo stress

Attenzione sulla soglia del 70%

La soglia `label_purity >= 0.70` e' una scelta di preprocessing supervisionato: usa la label vera del dataset per scartare finestre troppo miste. Va bene nel benchmark offline, ma in un'app reale non avremmo la label vera in tempo reale. Quindi non va presentata come pipeline real-time definitiva.

18. Struttura della presentazione finale

La consegna prevede una presentazione sintetica, massimo 15 slide. La struttura consigliata e' questa.

Slide	Titolo	Contenuto
1	Titolo e obiettivo	Stress detection da segnali wearable su WESAD
2	Dataset WESAD	Soggetti, sensori wrist/chest, task
3	Protocollo e label	Baseline, stress, amusement; mapping binario
4	Pipeline generale	Load.pkl -> windowing -> feature -> model -> metrics
5	Windowing e purezza	60s/30s, step, soglia 70%, finestre scartate
6	Feature extraction	Feature BVP/EDA/chest usate
7	Validazione	Leave-One-Subject-Out e motivo
8	BVP + kNN	Risultati e confronto paper
9	EDA + kNN	Risultati e confronto
10	Chest + RF	RESP, ECG, EMG
11	Sensor fusion	BVP+EDA o ECG+EMG+RESP
12	Confusion matrix	Errori principali
13	Discussione	Cosa funziona e perche'
14	Limiti	Dataset, laboratorio, feature semplici
15	Conclusione	Pipeline completa e possibili sviluppi

19. Organizzazione della consegna

La consegna deve essere ordinata e comprensibile per la docente. Non basta inviare script sparsi.

File/cartella	Contenuto	Obbligatorio?
src/	Script Python puliti	Si
data_features/	CSV delle feature generate	Consigliato
results/	CSV/txt con metriche	Si
figures/	Grafici usati nella presentazione	Si
presentation.pdf o pptx	Massimo 15 slide	Si
README.md	Come eseguire il progetto	Molto consigliato
requirements.txt	Librerie Python	Molto consigliato

README minimo

```
# WESAD Stress Detection Project

## Task
Binary classification: stress vs non-stress.

## Labels
1 baseline -> 0 non-stress
2 stress -> 1 stress
3 amusement -> 0 non-stress
0, 4, 5, 6, 7 excluded

## Run
python src/extract_bvp_features_all.py
python src/train_bvp_knn_loso.py

## Requirements
pip install -r requirements.txt
```

20. Checklist finale

Usate questa checklist per capire se il progetto e' davvero finito.

Area	Checklist
Ambiente	Python funziona, .venv attivo, requirements.txt creato
Dati	Tutti i soggetti validi caricati, S1/S12 esclusi
Feature	CSV finali creati per BVP, EDA, RESP, ECG, EMG
Label	Mapping binario corretto: 1/3 -> 0, 2 -> 1
Purezza	Soglia 70% applicata e documentata
Validazione	Leave-One-Subject-Out usato, non split casuale
Scaling	StandardScaler dentro ogni fold per kNN
Metriche	Accuracy, precision, recall, F1, confusion matrix
Risultati	Tabelle finali salvate e confrontate con paper
Grafici	Bar chart, confusion matrix, risultati per soggetto
Discussione	Spiegati risultati, differenze dal paper e limiti
Consegna	Codice + presentazione + README pronti

Ordine pratico da seguire da domani

- 1 Mettere ordine nella cartella: creare data_features, results, figures, src.
- 2 Spostare o copiare create_bvp_features_s2.py in src e trasformarlo in funzione generica.
- 3 Scaricare/mettere tutte le cartelle soggetto S2...S17 dentro data_raw.

- 4 Generare bvp_features_all_60s.csv su tutti i soggetti.
- 5 Fare controlli sul CSV: righe, soggetti, label, NaN, purezza.
- 6 Addestrare BVP + kNN con Leave-One-Subject-Out.
- 7 Salvare metriche e confusion matrix.
- 8 Ripetere la stessa procedura per EDA + kNN.
- 9 Passare a RESP + Random Forest, poi ECG e EMG.
- 10 Creare tabella finale e confronto con la Tabella 4 del paper.
- 11 Scrivere slide e discussione dei limiti.

Conclusione

Il progetto va completato in modo incrementale. La cosa piu' importante non e' provare subito tutti i segnali, ma rendere la pipeline robusta: stesso task, stesso mapping delle label, feature extraction chiara, nessun leakage, validazione per soggetto e metriche complete. Una volta completato BVP + kNN su tutti i soggetti, il resto del progetto diventa una replica controllata della stessa logica sugli altri segnali.

Fonti usate come riferimento

Fonte	Uso nel progetto
WESAD paper - Introducing WESAD, a Multimodal Dataset for Wearable Stress and Affect Detection	Dataset, sensori, protocollo, benchmark, Tabella 4
WESAD README	Struttura dei file, campionamenti, significato delle label
Linee guida progetto Interfacce Uomo-Macchina	Pipeline richiesta: preprocessing, filtering, feature extraction, classificatore, metriche oltre accuracy